MétéoSentinel

Météo, air et décisions personnalisées



6 services individuels
1 DataHub développé en commun

Une décision, plusieurs données

« Quand faire deux heures de vélo à Tunis ? »

Météo et prévisions

Qualité de l’air

Préférences personnelles

Un résultat expliqué

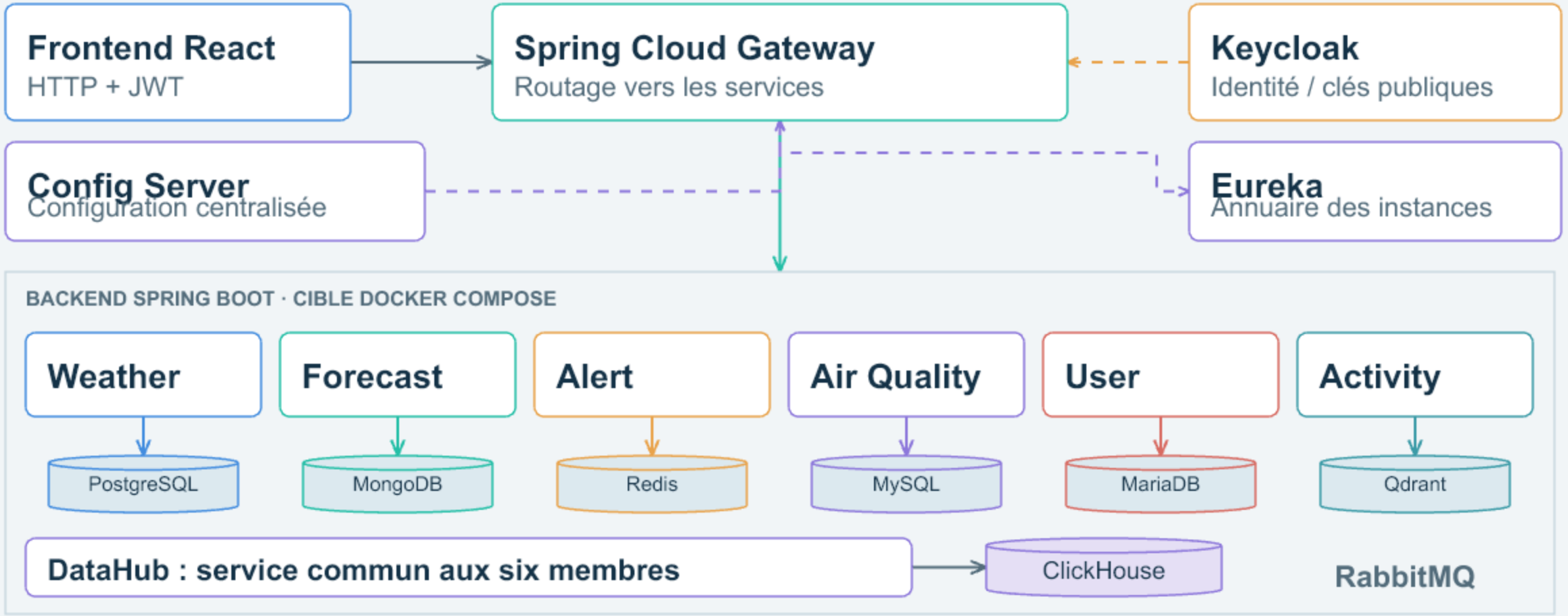
Créneaux classés, facteurs dominants
et données utilisées pour le calcul.

Les usages de la plateforme

Acteur	Ce qu’il peut faire
Visiteur	Consulter météo, prévisions et qualité de l’air
Utilisateur connecté	Gérer ses favoris et obtenir un classement personnalisé
Utilisateur connecté	Configurer les alertes et retirer son consentement
Administrateur	Administrer les règles et les rapports de qualité

Historique et provenance rendent chaque recommandation vérifiable.

Une architecture globale lisible



Une responsabilité par étudiant

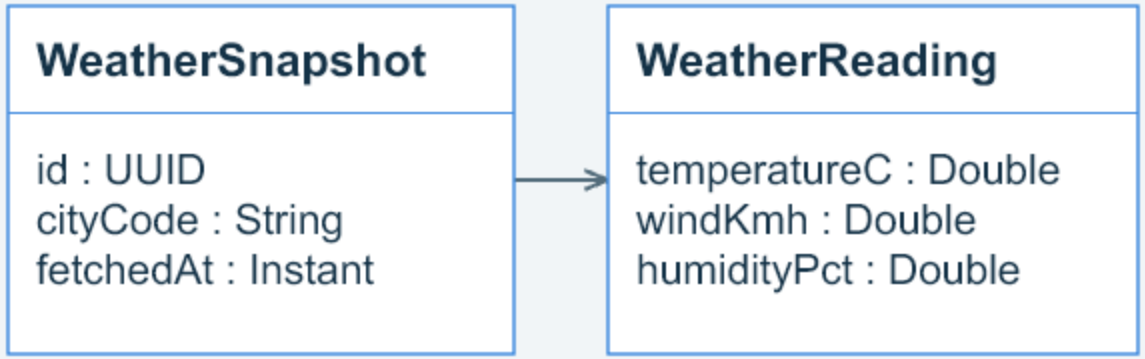
Responsable	Service individuel	Base exclusive
Nour Maroufi	Weather	PostgreSQL
Dhia Borzli	Forecast	MongoDB
Mohamed Hedi Bejaoui	Alert	Redis
Kaouther Halouti	Air Quality	MySQL
Ahmed Ben Mahmoud	User	MariaDB
Mohamed Aziz Sayhi	Activity	Qdrant (vectoriel)

DataHub : toute l’équipe. Présentation du service commun : Kaouther Halouti.

Weather Service

Nour Maroufi · PostgreSQL

Acquérir et historiser les conditions météo courantes.



CONTRATS REST

GET /api/weather/{city}

Réponses : 200, 400, 503

GET /api/weather/{city}/history

Réponses : 200, 400

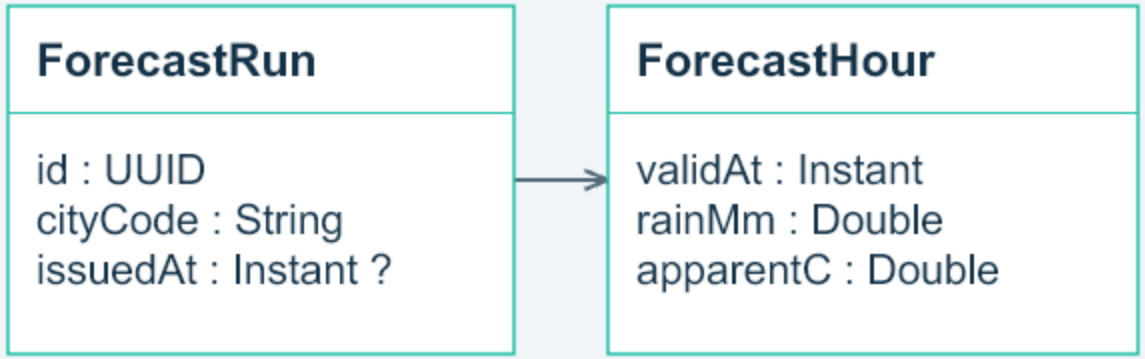
Synchrone Appelé par Activity et Alert

Asynchrone Publie WeatherUpdated

Forecast Service

Dhia Borzli · MongoDB

Servir les prévisions horaires et préserver leurs versions.



CONTRATS REST

GET /api/forecasts/{city}

Réponses : 200, 400, 404, 503

GET /api/forecasts/{city}/runs

Réponses : 200, 400, 503

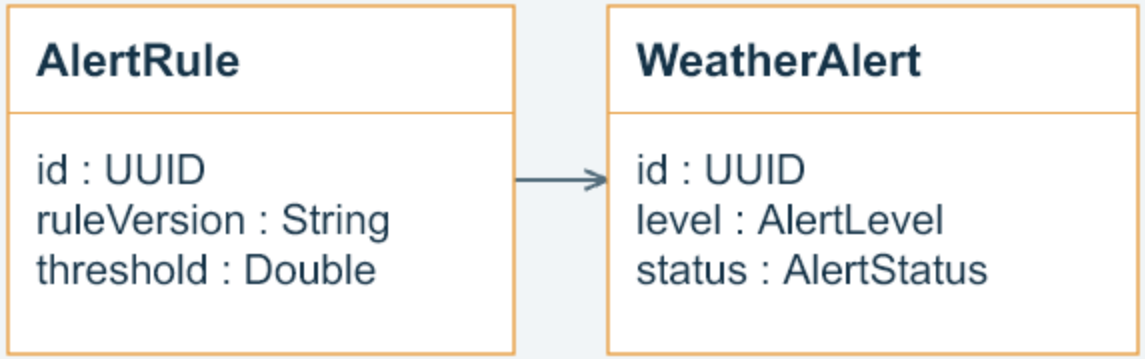
Synchrone Répond aux requêtes Activity

Asynchrone Publie ForecastUpdated

Alert Service

Mohamed Hedi Bejaoui · Redis

Classifier les situations et gérer le cycle de vie des alertes.



1 déclenche plusieurs

CONTRATS REST

GET /api/alerts/me

Réponses : 200, 401

POST /api/alerts/subscriptions

Réponses : 201, 400, 401, 409

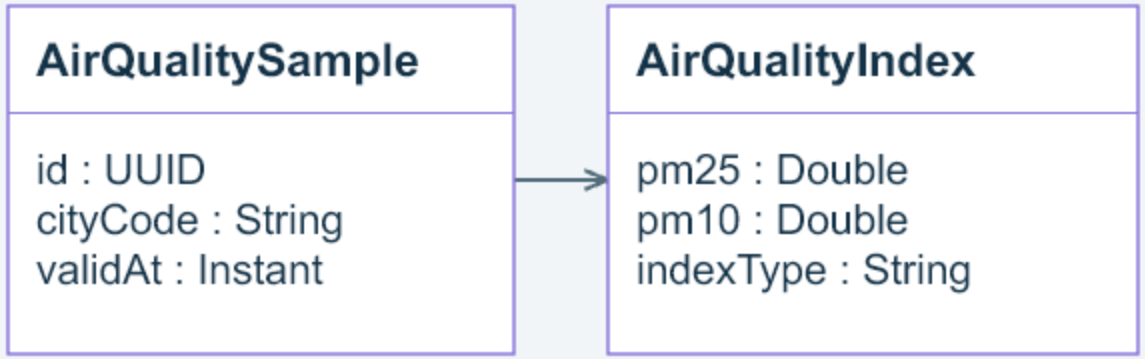
Synchrone Consulte User pour les consentements

Asynchrone Consomme WeatherUpdated / ForecastUpdated

Air Quality Service

Kaouther Halouti · MySQL

Fournir la pollution prévue et son indice documenté.



1 expose 1

CONTRATS REST

GET /api/air-quality/{city}

Réponses : 200, 400, 503

GET /api/air-quality/{city}/history

Réponses : 200, 400

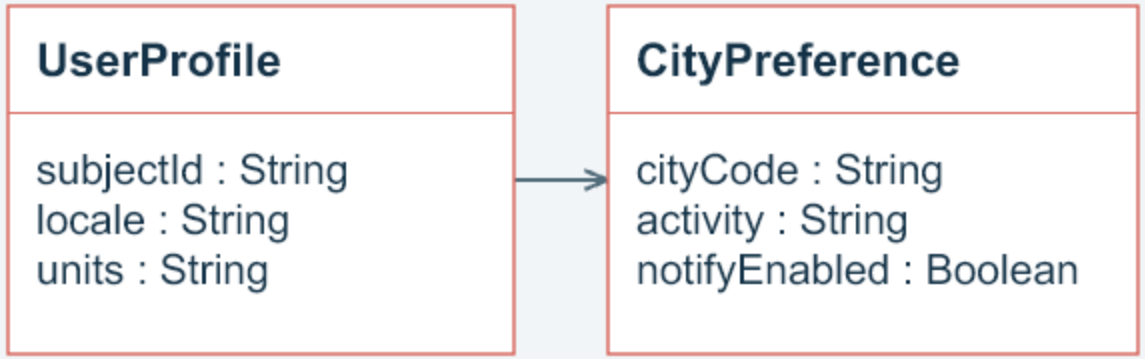
Synchrone Fournit un indicateur à Activity

Asynchrone Publie AirQualityUpdated

User Service

Ahmed Ben Mahmoud · MariaDB

Gérer profils métier, villes favorites et consentements.



CONTRATS REST

GET /api/users/me

Réponses : 200, 401, 404

PUT /api/users/me/preferences

Réponses : 200, 400, 401

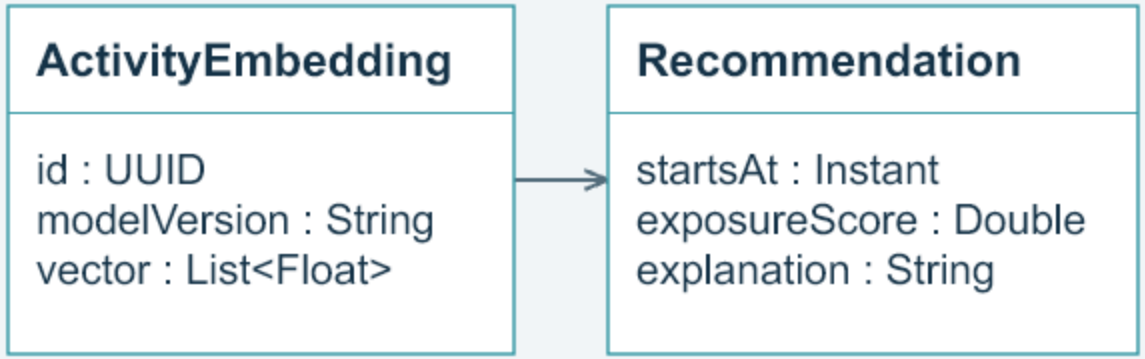
Synchrone Répond à Alert et Activity

Asynchrone Publie PreferencesChanged

Activity Service

Mohamed Aziz Sayhi · Qdrant (vectoriel)

Rechercher des activités et expliquer les créneaux adaptés.



Recherche puis contraintes numériques

CONTRATS REST

POST /api/activities/search

Réponses : 200, 400, 401, 503

POST /api/activities/recommendations

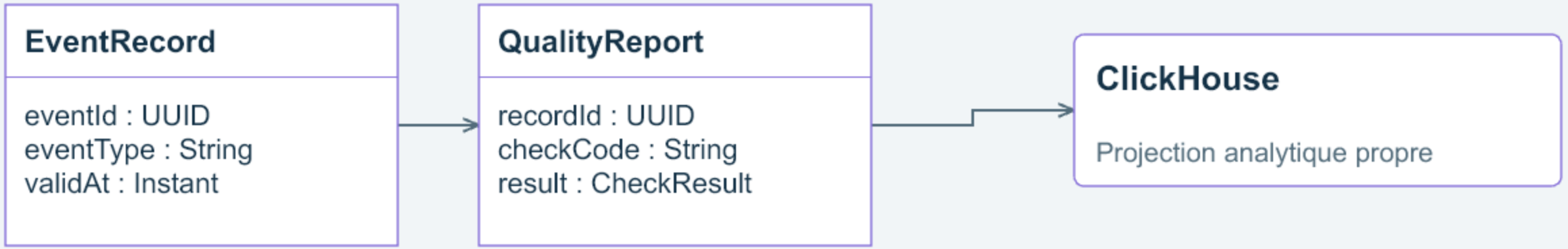
Réponses : 200, 400, 401, 422, 503

Synchrone Agrège Forecast, Weather, Air Quality et User

Asynchrone Publie RecommendationComputed

DataHub, le service commun

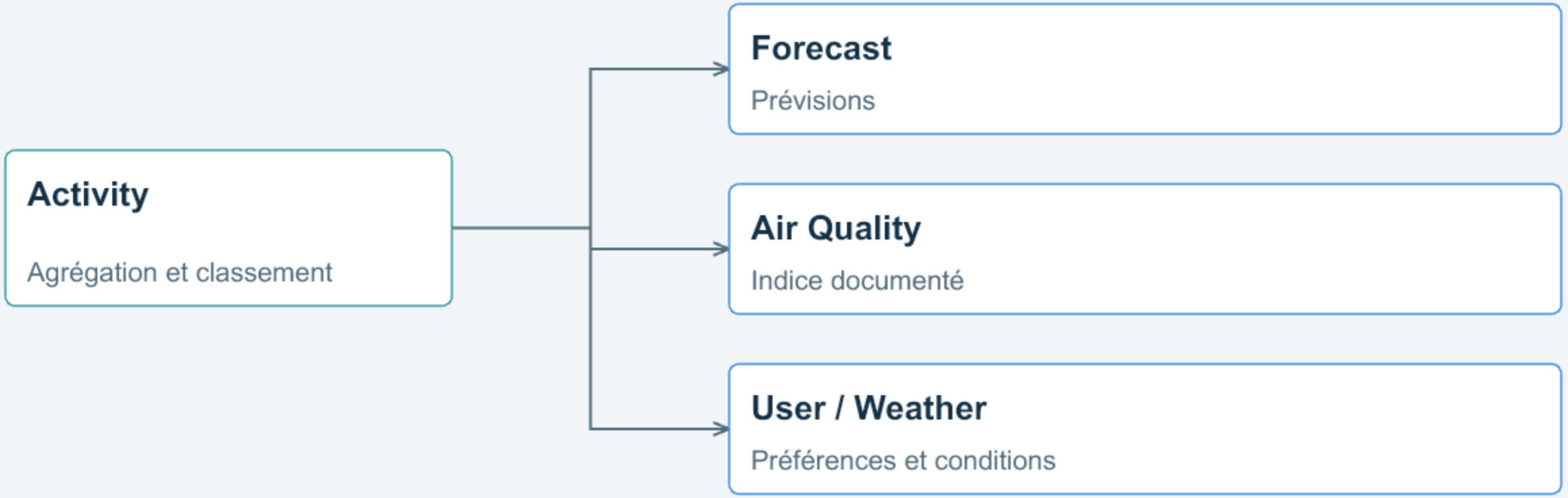
Historique, qualité et évaluation des prévisions



Entrées WeatherUpdated / ForecastUpdated / AirQualityUpdated

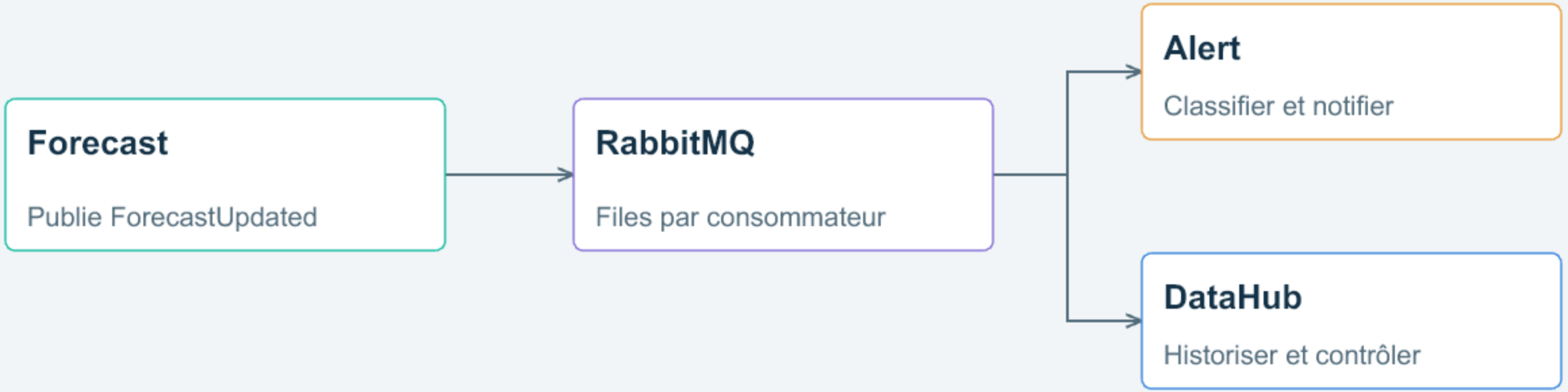
Sorties Rapports qualité et erreurs par ville, variable et horizon

Une recommandation par OpenFeign



Une donnée indispensable manque : réponse explicite, pas de score inventé.

Une mise à jour sans attente couplée



eventId stable

Retry borné et quarantaine

Une livraison répétée doit garder un seul effet métier.

Identité et découverte des services



Le JWT accompagne les requêtes métier. Eureka ne transporte pas leur contenu.

Classification et classement

Type	Origine	Usage
Indice de qualité de l’air	Échelle du fournisseur documentée	Entrée conservée avec provenance
Classe d’exposition météo	Règles du projet, versionnées	Explication dans Alert / Activity
Classement des créneaux	Pire heure puis moyenne	Sélection des fenêtres continues

DataHub conserve les versions et vérifie la qualité des données.

Aucune précision de modèle ni garantie de sécurité n’est annoncée.

Git et suivi du travail

Backend / Frontend / Documentation / README



GitHub Issues + jalons

Tickets par service, critères de recette et revues.
Chaque membre doit produire ses vrais commits personnels.

Un planning lié aux séances

S4	S5–S6	S7	S8	S9–S10
Conception	Services et REST	Événements	Sécurité	Docker et recette

Données de test et contrats stables avant les développements parallèles.

Les risques et leurs réponses

Risque	Réponse prévue
API indisponible ou données manquantes	Cache, timeout et état incomplet visible
Sept services difficiles à intégrer	Contrats stabilisés et intégration par étapes
Messages dupliqués ou en erreur	Idempotence, retry borné et quarantaine
Confusion sur la précision des scores	Provenance, version et validation des règles

Chaque membre doit pouvoir expliquer aussi les autres parties du dossier.

La conception à faire valider

Préparé pour S4	À réaliser par l'équipe
Projets Spring Boot compilables et démarrables	Relire, modifier et committer personnellement
Modèles, API, bases et scénarios de panne	Confirmer le découpage 6 + 1 avec l'enseignant
Gateway, Eureka et Config Server locaux	Tester Docker et l'intégration après validation
Support et plan oral de 10 minutes	Présenter chacun son service et répondre

Le métier commence après validation. Aucun produit terminé n'est annoncé.

Sources et notes pour la discussion

Consignes primaires

Grille Validation 1 + rendu détaillé + consignes Blackboard

Architecture et sources techniques

Conversation partagée, Spring Cloud, Keycloak,
Open-Meteo et RabbitMQ

Références complètes, hypothèses et scripts oraux dans le dossier technique.